Lab 02 - Identifying and Preventing Phishing

Attacks

Student: Nenad Damjanović | Environment: Kali Linux (VirtualBox)

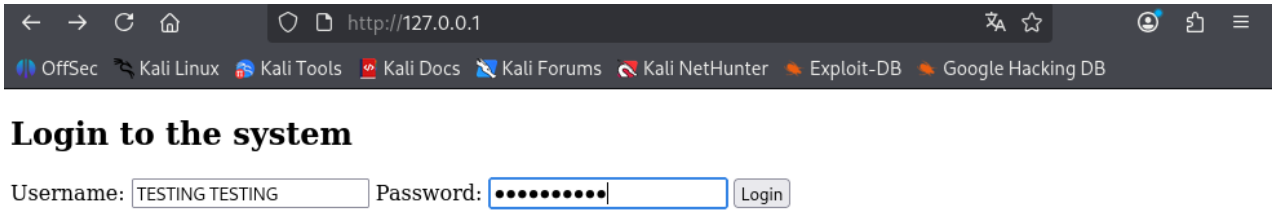
Objective and setup

Demonstrate how a cloned login form can collect submitted test parameters in an authorized, isolated laboratory. An original HTML page was served at 127.0.0.1:8000/login.html; Social-Engineer Toolkit (SET) served its local clone at 127.0.0.1:80. Only fictional credentials were used.

Procedure

1. Created a local HTML login form and started the Python HTTP server.
2. Opened SET: Social-Engineering Attacks → Website Attack Vectors → Credential Harvester → Site Cloner.
3. Entered local POST-back IP 127.0.0.1 and cloned the local test URL.
4. Freed port 80 from an earlier Python test server, then opened the cloned page and submitted dummy data.

Figure 1. Cloned local login page in Firefox

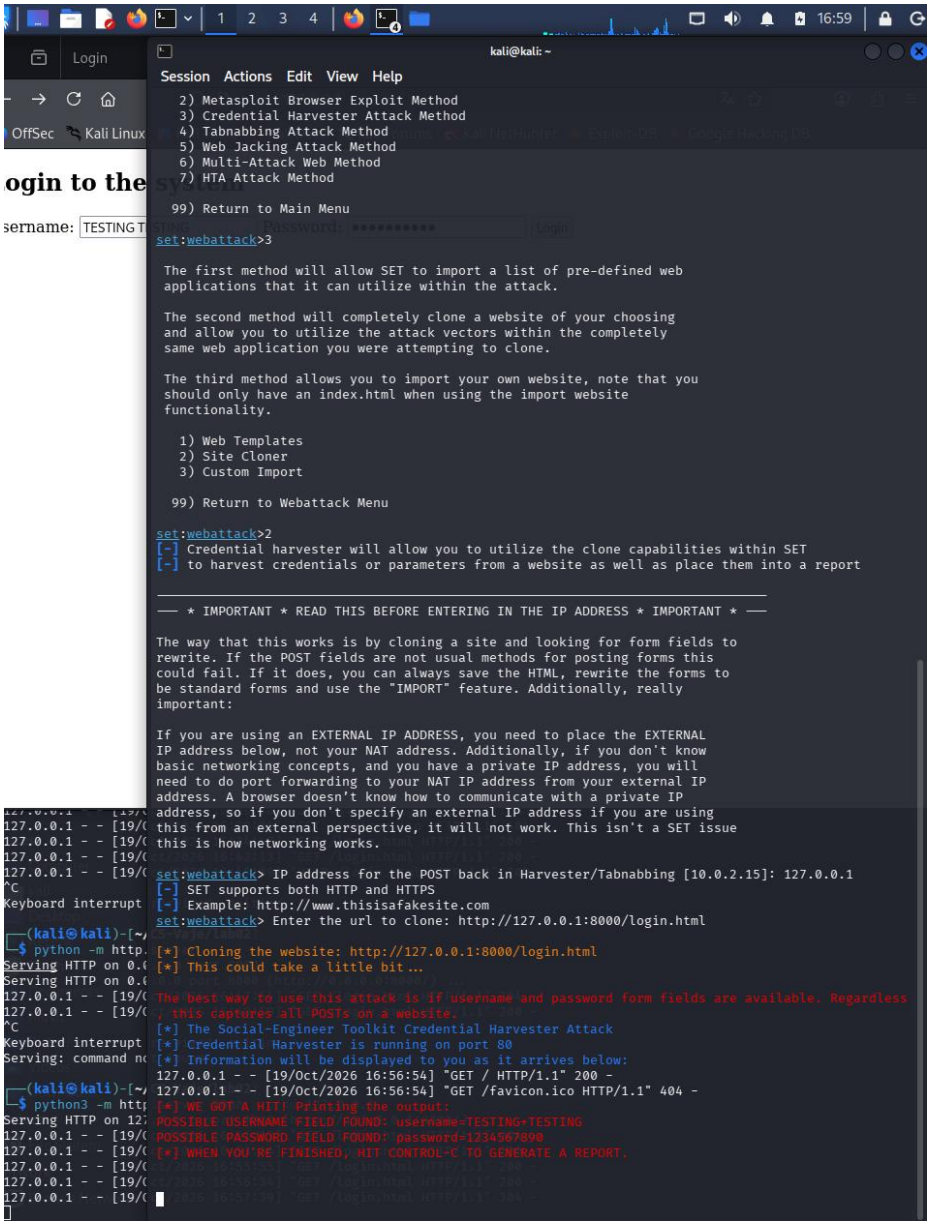


Observed result

The browser showed the expected username/password form at the SET address. Submitting the form generated an HTTP POST that SET recorded. This illustrates credential-harvesting behavior without involving external websites.

Credential harvesting result

Figure 2. SET terminal showing “WE GOT A HIT!” and fictional username/password parameters.



Questions and answers

How can phishing be recognized? Inspect the actual domain, unexpected sign-in requests, misleading links, suspicious urgency, and visual inconsistencies. A realistic-looking page is not proof of legitimacy.

How can users protect themselves? Navigate through trusted bookmarks, check the complete URL, use password managers and phishing-resistant MFA/passkeys, and report suspicious messages.

Conclusion: The local simulation demonstrated how a cloned form can capture test login values. Testing was confined to the student-controlled VM.

**ChatGPT is used to create a report from my own work and screenshots*